

Análise de Algoritmos Iterativos

Contagem de execuções de cada instrução II

Exercícios

Anote logo todos os enunciados.
(só precisa anotar os pseudocódigos)

Exercício: contagem de execuções

O algoritmo abaixo recebe como entrada uma lista A , contendo n elementos.

```
1 soma = 0
2 for i in range(n):
3     soma = soma + A[i]
4     for j in range(i, n):
5         soma = soma + A[j]
6         for k in range(j - i):
7             soma = soma + A[k]
8 return soma
```

Linha	Número de execuções
1	$T_1(n) = 1$
2	$T_2(n) = n + 1$
3	$T_3(n) = n$
4	$T_4(n) = \sum_{i=0}^{n-1} (n - i + 1)$
5	$T_5(n) = \sum_{i=0}^{n-1} (n - i)$
6	$T_6(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1) = \sum_{i=0}^{n-1} \sum_{\ell=0}^{n-1-i} (\ell + 1)$
7	$T_7(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i) = \sum_{i=0}^{n-1} \sum_{\ell=0}^{n-1-i} \ell$
8	$T_8(n) = 1$

Exercício: Controle de Lotação do Ônibus

Considere uma lista L de tamanho n contendo movimentações de passageiros e um inteiro C (capacidade do ônibus). Forneça contagem de execuções (pior caso).

```
1 def controle_onibus(L, C):
2     n = len(L)
3     i = 0
4     passageiros = 0
5     while i < n and passageiros < 2 * C:
6         M = L[i]
7         passageiros += M
8         if passageiros >= 2 * C:
9             print("hora de partir")
10        i += 1
```

Gabarito: Tabela de Frequência no Pior Caso

Linha	Instrução	Frequência (Pior Caso)
1	<code>def controle_onibus(L, C):</code>	1
2	<code>n = len(L)</code>	1
3	<code>i = 0</code>	1
4	<code>passageiros = 0</code>	1
5	<code>while i < n and passageiros < 2*C:</code>	$n + 1$
6	<code>M = L[i]</code>	n
7	<code>passageiros += M</code>	n
8	<code>if passageiros >= 2 * C:</code>	n
9	<code>print("hora de partir")</code>	1
10	<code>i += 1</code>	n

Gabarito: Justificativa do Pior Caso

Definição do Pior Caso

O pior caso em termos de número total de instruções executadas ocorre quando:

1. O laço **não encerra prematuramente**: a lista é percorrida por completo (n iterações válidas), o que exige $n + 1$ testes na linha 5.
2. A cadeia condicional testa a **maior quantidade de verificações**:
 - ▶ A condição da linha 8 avalia como falso em todas as iterações (executando n vezes o teste).
 - ▶ O algoritmo encerra quando é “hora de partir” ou quando a lista termina, portanto, no pior caso, o print é executado apenas uma vez.

Observação sobre a Linha 5

O laço `while` executa n vezes com a condição avaliada como verdadeira e realiza 1 teste adicional ao término (quando $i = n$), totalizando $n + 1$ execuções.

Análise em Função do Número de Bits

- ▶ Em muitos algoritmos numéricos, o número de passos não depende do tamanho de estruturas (como vetores ou matrizes), mas do **valor numérico** das variáveis de entrada.
- ▶ Na Teoria da Complexidade, o tamanho da entrada (n) é a **quantidade de bits** necessários para representá-la.
- ▶ Um número inteiro positivo X representado com até b bits possui limite superior dado por:

$$X < 2^b \quad \text{ou} \quad X \leq 2^b - 1$$

- ▶ Para a análise de **pior caso**, identificamos a combinação de valores que maximiza as iterações do laço e expressamos essa contagem como uma função de 2^b (tempo pseudo-polinomial / exponencial em b).

Exercício: O Pulo do Sapo no Poço

Considere inteiros positivos P (profundidade), S (salto) e E (escorregamento), com $S > E$, todos representados com no máximo b bits ($P, S, E \leq 2^b - 1$).

```
1 def posicoes_sapo(P, S, E):  
2     altura = 0  
3     while altura + S < P:  
4         altura += S  
5         print(altura)  
6         altura -= E  
7     altura += S  
8     print(altura)
```

Questão: Determine a frequência de execução de cada linha no **pior caso**, expressando a contagem de iterações em função de b bits.

Gabarito: Tabela de Frequência no Pior Caso

No pior caso, $P = 2^b - 1$, $S = 1$ e $E = 0$ (avança apenas 1 unidade por iteração). Nas iterações do while: $0 \leq \text{altura} < P - S = 2^b - 2$. Ou seja, $2^b - 2$ iterações.

Linha	Instrução	Frequência (Pior Caso em b bits)
1	<code>def posicoes_sapo(P, S, E):</code>	1
2	<code>altura = 0</code>	1
3	<code>while altura + S < P:</code>	$2^b - 1$
4	<code>altura += S</code>	$2^b - 2$
5	<code>print(altura)</code>	$2^b - 2$
6	<code>altura -= E</code>	$2^b - 2$
7	<code>altura += S</code>	1
8	<code>print(altura)</code>	1

Obs.: Para evitar overflow de inteiros no i na última iteração, assumamos que b é menor que o máximo de bits do tipo inteiro.

Exercício Proposto: Laço com Crescimento Exponencial em Bits

Considere um inteiro $n \geq 1$ representado por no máximo b bits ($n \leq 2^b - 1$). Analise o algoritmo abaixo, onde o índice de controle i inicia em 1 e é duplicado a cada iteração.

```
1 def soma_potencias(n):  
2     i = 1  
3     soma = 0  
4     while i < n:  
5         soma += i  
6         i = i * 2  
7     return soma
```

Dica: Assuma que $i = 2^k$, para $k = 0, \dots, b - 1$.

Obs.: Note que k não pode valer b , pois isso tornaria $i = 2^b > 2^b - 1$ (overflow de inteiros com b bits).

Gabarito: Exercício Proposto (em Função de b Bits) I

Linha	Instrução	Frequência (Pior Caso em b bits)
1	<code>def soma_potencias(n):</code>	1
2	<code> i = 1</code>	1
3	<code> soma = 0</code>	1
4	<code> while i < n:</code>	$b + 1$
5	<code> soma += i</code>	b
6	<code> i = i * 2</code>	b
7	<code> return soma</code>	1

Notas explicativas

- ▶ Valor de k no início de cada iteração: $0, 1, \dots, b - 1$. Total de b iterações.
- ▶ O `while` tem uma iteração a mais: $b + 1$ iterações.
- ▶ Note que são **poucas iterações**: depende de b , não de 2^b .
 - ▶ Como $n_{\max} = 2^b - 1$, temos que $b = \log_2(n_{\max} + 1)$.